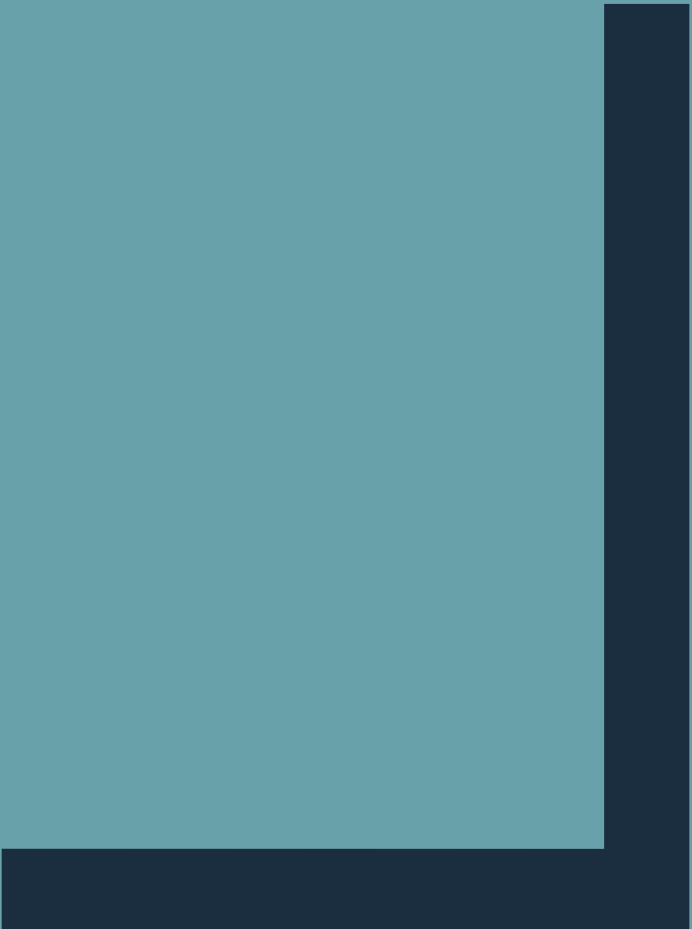
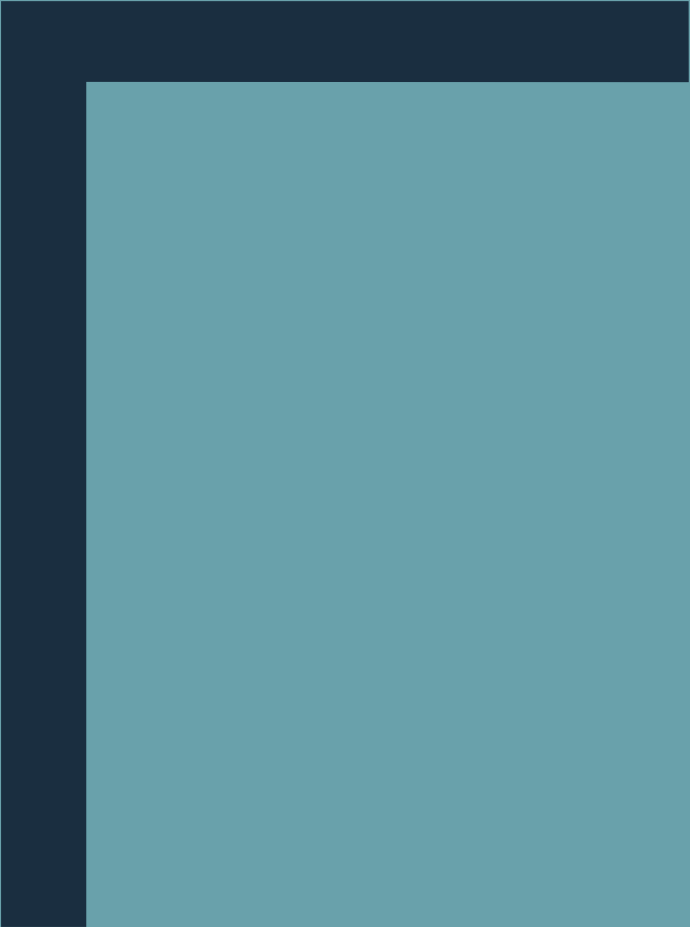




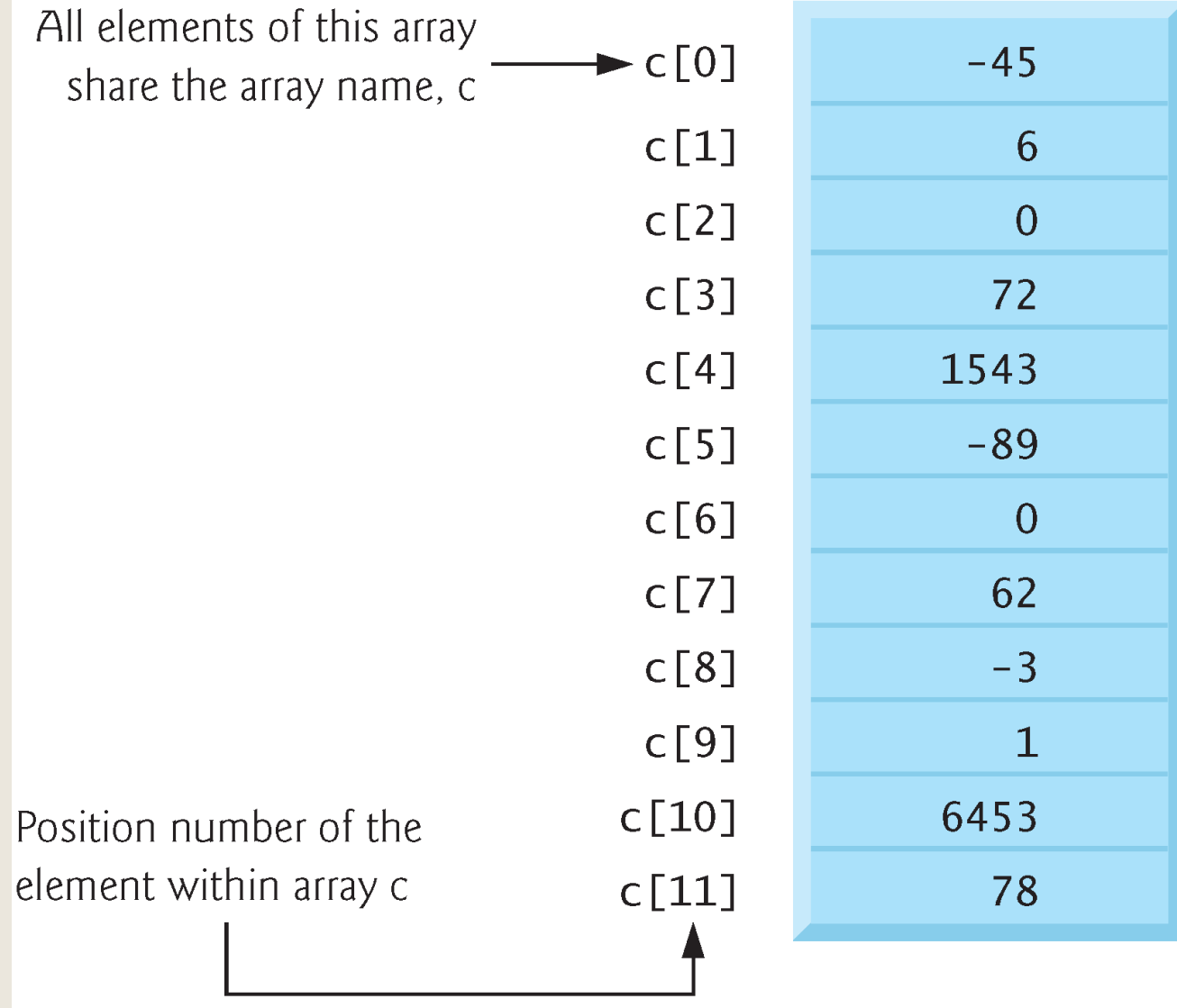
ARRAYS I

Arrays

- Arrays are data structures consisting of related data items of the same type.
- An array is a group of contiguous memory locations that all have the same type.
- To refer to a particular location or element in the array, we specify the array's name and the position number of the particular element in the array.

Arrays

- This figure shows an integer array called `c`, containing 12 elements.



Arrays

- Any one of these elements may be referred to by giving the array's name followed by the position number of the particular element in square brackets ([]).
- The first element in every array is the **zeroth element**.
- An array name, like other identifiers, can contain only letters, digits and underscores and cannot begin with a digit.
- The position number within square brackets is called an **index** or **subscript**.
- An index must be an integer or an integer expression.

Arrays

- Any one of these elements may be referred to by giving the array's name followed by the position number of the particular element in square brackets ([]).
- The first element in every array is the **zeroth element**.
- An array name, like other identifiers, can contain only letters, digits and underscores and cannot begin with a digit.
- The position number within square brackets is called an **index** or **subscript**.
- An index must be an integer or an integer expression.

Arrays

- For example, if $a = 5$ and $b = 6$, then the statement
 $c[a + b] += 2;$
adds 2 to array element $c[11]$.
- An indexed array name is an **lvalue**—it can be used on the left side of an assignment

Arrays

- Let's examine array `c` more closely.
- The array's name is `c`.
- Its 12 elements are referred to as `c[0]`, `c[1]`, `c[2]`, ..., `c[10]` and `c[11]`.
- The value stored in `c[0]` is `-45`, the value of `c[1]` is `6`, `c[2]` is `0`, `c[7]` is `62` and `c[11]` is `78`.
- To print the sum of the values contained in the first three elements of array `c`, we'd write

```
printf("%d", c[0] + c[1] + c[2]);
```

Arrays

- The brackets used to enclose the index of an array are actually considered to be an operator in C.
- They have the same level of precedence as the function call operator (i.e., the parentheses that are placed after a function name to call that function).

Operators	Associativity	Type
[] () ++ (<i>postfix</i>) -- (<i>postfix</i>)	left to right	highest
+ - ! ++ (<i>prefix</i>) -- (<i>prefix</i>) (<i>type</i>)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Defining Arrays

- Arrays occupy space in memory.
- You specify the type of each element and the number of elements each array requires so that the computer may reserve the appropriate amount of memory.
- The following definition reserves 12 elements for integer array c, which has indices in the range 0-11.

```
int c[12];
```

Defining Arrays

- The definition

```
int b[100], x[27];
```

reserves 100 elements for integer array b and 27 elements for integer array x.

- These arrays have indices in the ranges 0–99 and 0–26, respectively.
- Arrays may contain other data types.
- Character strings and their similarity to arrays and the relationship between pointers and arrays will be discussed in next chapters.

Array Examples



```
#include <stdio.h>
int main(void)
{
    int n[5];
    int i;
    for (i = 0; i < 5; i++)
        n[i] = 0;
    printf("Element \t Value\n");
    for (i = 0; i < 5; i++)
        printf("%7d \t %5d\n", i, n[i]);
    return 0;
}
```

Output:

Element	Value
0	0
1	0
2	0
3	0
4	0

Array Examples

- The elements of an array can also be initialized when the array is defined by following the definition with an equals sign and braces, {}, containing a comma-separated list of **array initializers**.

Array Examples



```
#include <stdio.h>
int main(void)
{
    int n[5] = {2,4,6,8,10};

    printf("Element \t Value\n");
    for (int i = 0; i < 5; i++)
        printf("%7d \t %5d\n", i, n[i]);
    return 0;
}
```

Output:

Element	Value
0	2
1	4
2	6
3	8
4	10

Array Examples

- If there are fewer initializers than elements in the array, the remaining elements are initialized to zero.
- For example, the elements of the array `n` could have been initialized to zero as follows:

```
int n[10] = {0};
```

- This explicitly initializes the first element to zero and initializes the remaining nine elements to zero because there are fewer initializers than there are elements in the array.

Array Examples

- It's important to remember that arrays are not automatically initialized to zero.
- You must at least initialize the first element to zero for the remaining elements to be automatically zeroed.
- The array definition

```
int n[5] = {32, 27, 64, 18, 95, 14};
```

causes a syntax error because there are six initializers and only five array elements.

Array Examples

- If the array size is omitted from a definition with an initializer list, the number of elements in the array will be the number of elements in the initializer list.
- For example,

```
int n[ ] = {1, 2, 3, 4, 5};
```

would create a five-element array initialized with the indicated values